

Tutorial 4 | OS Monsoon'25

Section A

| Author → [Rachit Bhandari](#)

System Calls in Operating Systems



What is a System Call?

A **system call** is the interface between a **user program** and the **operating system kernel**.

When an application needs to perform an operation that requires **hardware access** or **privileged resources**, it cannot directly interact with hardware (for safety). Instead, it asks the OS through a system call.

Examples

- Opening a file → `open()`
- Reading from a file → `read()`
- Creating a process → `fork()`
- Exiting a program → `exit()`

Analogy

Think of the OS as a **restaurant kitchen** →

- Customers (user programs) sit in the dining area (user space).
- They cannot walk into the kitchen (kernel space).
- Instead, they place orders through waiters (system calls).
- The chef (kernel) prepares the meal (operation) and sends it back.

So, **system calls are the waiters that carry requests safely to the kitchen.**

User Space vs Kernel Space

Operating systems divide memory into two zones to enforce **protection** and **stability**.

User Space

- Where application code, utility software runs.
- Non-privileged mode → cannot directly access hardware or kernel memory.
- If a program crashes, only that process is affected.
- Must make system calls to the kernel to request access.

Kernel Space

- Like the head of the family (decides who will get how much freedom).
- Where the operating system core runs.
- Privileged mode → can control CPU, memory, devices (system hardware).
- If something goes wrong here, the **whole system may crash** (kernel panic).

Why Do We Use System Calls?

Safety & Security

- Prevents malicious or buggy apps from directly controlling hardware.

Abstraction

- Programs don't need to know hardware details (e.g., `read()` works for files, disks, network sockets alike).

Portability

- Same code (`printf()` , `open()`) works on different machines.

Resource Management

- Kernel enforces fair usage of CPU, memory, I/O.

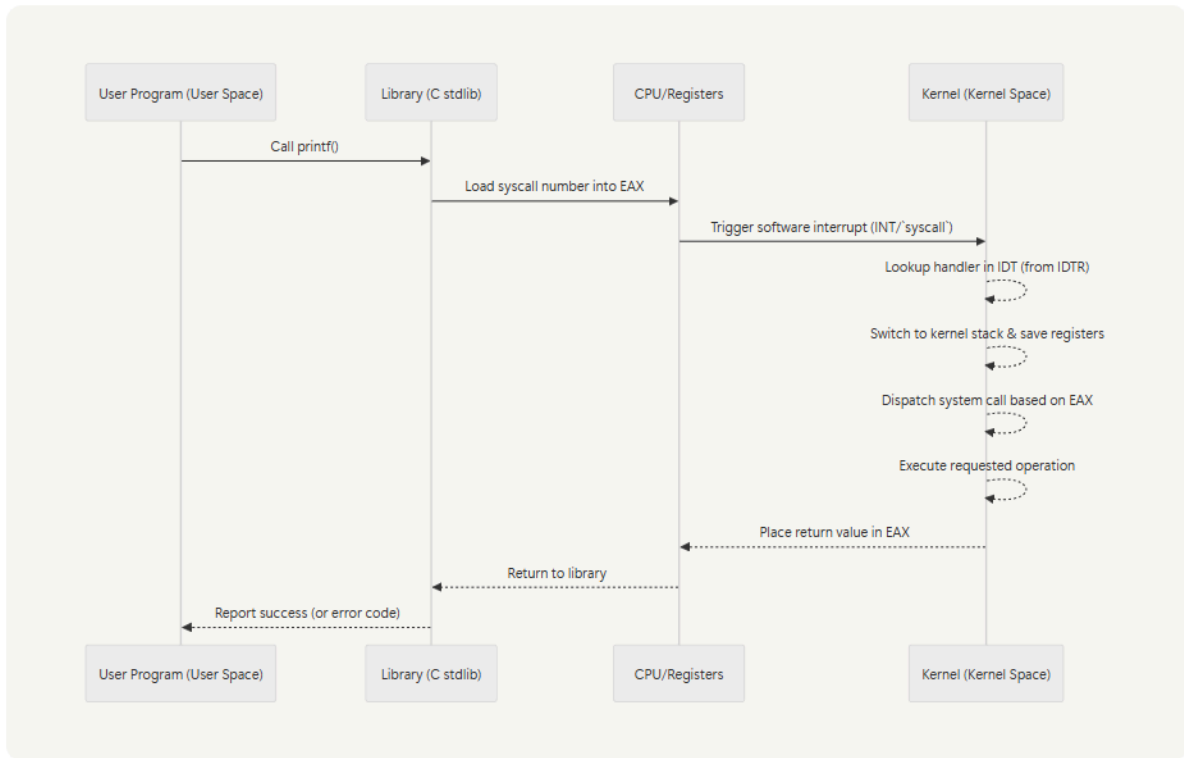
Inter-Process Communication

- System calls provide mechanisms for processes to communicate and synchronize with each other.

Steps for Making a System Call

- **Library Call → System Call**
 - Example: `printf()` eventually calls `write()` to output to the terminal.
 - Each system call has a unique number (declared in `unistd.h`).
 - The syscall number is placed in the **EAX register**.
- **User Mode → Kernel Mode Switch**
 - At boot, the kernel sets up the **Interrupt Descriptor Table (IDT)**, whose address is stored in the **IDTR** register (per CPU).
 - x86 systems have **256 IDT entries**, each pointing to an interrupt handler.
 - A **software interrupt (trap)** is triggered.
 - CPU switches to **kernel stack**.
 - Registers are saved on the kernel stack (to restore later).
 - The syscall number from **EAX** is used to find and execute the requested syscall.
- **Return to User Mode**
 - Kernel finishes the operation.
 - Result (success/error code) is returned to the user program.
 - Execution continues in **user mode**.

Visual Flow



Example → System Call in Action

C Program

```

#include <unistd.h>
#include <sys/syscall.h>

int main() {
    const char msg[] = "Hello via write system call!\n";
    syscall(SYS_write, 1, msg, sizeof(msg) - 1);
    // SYS_write = 1 (first system call)
    // you can even try replacing it with 1 inside the method
    return 0;
}

```

Explanation

- `syscall()` directly invokes a system call.
- `SYS_write` = system call number for `write`.
- Arguments:

- `1` → file descriptor (stdout)
- `msg` → buffer to print
- `sizeof(msg)-1` → number of bytes

This program **skips** `printf()` and directly talks to the kernel.

Fork



Introduction to Fork

- **Definition** → `fork()` is a system call used to create a new process in Unix/Linux systems.
- **Purpose** → To create a child process that runs concurrently with the parent process.
- **Key Point** → `fork()` is fundamental for multitasking and process management.

How It Works

- **Process Creation** → When `fork()` is called, a new process (child) is created.
- **Return Values** →
 - `0` in the child process.
 - `Child PID` in the parent process.

- `-1` on error.
- **Memory Sharing** → Initially, parent and child share memory, but changes are handled with Copy-On-Write (COW).

Questions

Question 1

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    printf("OS is a fundamental CS course!\n");
    return 0;
}
```

▼ Output

```
OS is a fundamental CS course!
OS is a fundamental CS course!
```

Question 2

```
#include <stdio.h>
#include <unistd.h>
int main() {
    for (int i = 0; i < 2; i++) {
        fork();
        printf("Hello!\n");
    }
    return 0;
}
```

▼ Output

```
Hello!  
Hello!  
Hello!  
Hello!  
Hello!  
Hello!
```

Question 3

```
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    if (fork() || fork())  
        fork();  
    printf("Hi!\n");  
    return 0;  
}
```

▼ Output

```
Hi!  
Hi!  
Hi!  
Hi!  
Hi!
```

